

CSE 106: Data Structures and Algorithms I

Sessional

Online on Binary Search Tree (BST)

Time: 35 minutes + 3 minutes for submission

Marks: 10

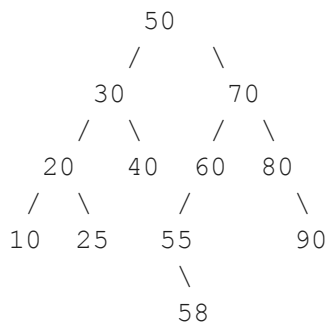
Problem Description

Given a Binary Search Tree (BST) and two keys, find their Lowest Common Ancestor (LCA).

Definition of Lowest Common Ancestor (LCA):

The Lowest Common Ancestor of two nodes with keys `key1` and `key2` in a binary tree is the deepest (lowest) node that has both `key1` and `key2` as descendants (where we allow **a node to be a descendant of itself**).

Visual Example:



- $LCA(20, 40) = 30$ (node 30 is the lowest common ancestor of 20 and 40)
- $LCA(10, 40) = 30$ (node 30 is the lowest common ancestor of 10 and 40)
- $LCA(10, 90) = 50$ (node 50 is the lowest common ancestor of 10 and 90)
- $LCA(30, 40) = 30$ (node 30 is ancestor of itself and 40)
- $LCA(58, 90) = 70$ (node 70 is the lowest common ancestor of 58 and 90)

Complexity Requirements:

- **100% marks:** Constant space $O(1)$ and $O(\text{height})$ time complexity solution
- **70% marks:** Any other correct solution (e.g., using extra space or higher time complexity)

Task

You need to add the `find_lca()` method to both the abstract base class and its implementation.

Add the following pure virtual method declaration to the `BST` class:

```
/**
 * Find the Lowest Common Ancestor of two keys in the BST
 * @param key1 - First key
 * @param key2 - Second key
 * @return The key of the LCA node
 * Assumption: Both keys are guaranteed to exist in the BST
 */
virtual Key find_lca(Key key1, Key key2) = 0;
```

Add the following method to your `ListBST` class:

```
/**
 * Find the Lowest Common Ancestor of two keys in the BST
 * @param key1 - First key
 * @param key2 - Second key
 * @return The key of the LCA node
 * Assumption: Both keys are guaranteed to exist in the BST
 */
Key find_lca(Key key1, Key key2) override;
```

Note: You can assume that both keys will always exist in the BST. No error handling for missing keys is required.

Main Function

```
#include "BST.hpp"
#include "listBST.hpp"
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream input("in_lca.txt");

    if (!input.is_open()) {
        cerr << "Error opening file!" << endl;
        return 1;
    }

    int n;
    input >> n;

    BST<int, string>* bst = new ListBST<int, string>();

    // Read and insert keys
    for (int i = 0; i < n; i++) {
        int key;
        string value;
        input >> key >> value;
        bst->insert(key, value);
    }

    // Read number of queries
    int q;
    input >> q;

    // Process each query
    for (int i = 0; i < q; i++) {
        int key1, key2;
        input >> key1 >> key2;

        int lca = bst->find_lca(key1, key2);
        cout << "LCA(" << key1 << ", " << key2 << ") = " << lca << endl;
    }

    delete bst;

    input.close();

    return 0;
}
```

Input Format

```
n
key1 value1
key2 value2
...
keyn valuen
q
query_key1_1 query_key1_2
query_key2_1 query_key2_2
...
query_keyq_1 query_keyq_2
```

Where:

- n is the number of key-value pairs to insert into the BST
- Each of the next n lines contains a key (integer) and its associated value (string)
- q is the number of LCA queries
- Each of the next q lines contains two keys for which to find the LCA

Output Format

$\text{LCA}(\text{key1}, \text{key2}) = \text{lca_key}$

For each query, output the LCA of the two given keys.

Sample Input 1

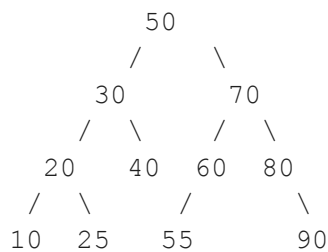
```
11
50 apple
30 banana
70 cherry
20 date
40 elderberry
60 fig
80 grape
10 mango
25 orange
55 kiwi
90 melon
```

6
10 25
20 40
10 40
10 90
30 40
60 80

Sample Output 1

LCA(10, 25) = 20
LCA(20, 40) = 30
LCA(10, 40) = 30
LCA(10, 90) = 50
LCA(30, 40) = 30
LCA(60, 80) = 70

Explanation: The BST structure is:



- LCA(10, 25): Both are children of 20, and 20 is their common ancestor
- LCA(20, 40): Both are in left subtree of 50, and 30 is their common ancestor
- LCA(10, 40): Both are in left subtree of 50, deepest level, 30 is their common ancestor
- LCA(10, 90): One in left subtree, one in right subtree, so root (50) is LCA
- LCA(30, 40): 30 is ancestor of itself and 40
- LCA(60, 80): Both are in right subtree of 50, and 70 is their common ancestor